8-3 · 수집→Lake→Spark→VectorDB 흐름 자동화 · 이론 1H

파이프라인 자동화 설계

어디서 자르나, 무엇을 넘기나, 어디서 부르나

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① 파이프라인 구간을 재시도 가능한 태스크 단위로 자르는 기준을 말할 수 있다

② 태스크 사이에 XCom 이 아니라 스토리지 경로를 넘기는 이유를 설명한다

③ 같은 도커 네트워크의 컨테이너끼리는 컨테이너 이름이 주소임을 안다

④ SparkSubmitOperator 제출과 파이썬 태스크의 DAG 모양이 같음을 안다

다음 1.5H 실습에서 7주차 컨테이너를 Airflow 네트워크에 붙이고 앞 태스크를 채웁니다

1. 오늘 만들 흐름

7주 동안 손으로 돌리던 다섯 구간을 DAG 하나로

수집	Lake 적재	검증	임베딩	VectorDB
collect 투입구 → MinIO	raw-docs 원본 그대로	validate 나쁜 문서 막기	build_kb.py 7주차 모델 그대로	Qdrant lecture_kb

실패했을 때 그 단계만 다시 돌리고 싶은 지점이 곧 태스크 경계

자동화 설계의 핵심은 「어디서 자르느냐」 — 전체를 함수 하나에 넣으면 중간 실패 때 처음부터 다시

어디서 자르나 — 네 가지 기준

1. **실패·재시도 단위로**
   택배의 집하 → 분류 → 배송처럼 단계마다 완료가 기록돼야 문제 지점이 보인다

2. **비싼 일과 싼 일을 섞지 않는다**
   임베딩(비용)과 파일 복사(무료)를 한 태스크에 두면 재시도가 비싸진다

3. **외부 시스템이 바뀌는 지점에서**
   API 호출·스토리지 쓰기·DB 적재는 각각 다른 이유로 실패한다

4. **너무 잘게 나누지 않는다**
   태스크 하나마다 실행 오버헤드가 있고 화면도 복잡해진다

이 파이프라인의 태스크 — 실패 원인과 재시도 비용

태스크	실패 원인	재시도가 싼가
수집(collect)	원천 API 장애·네트워크	싸다 — 재시도 유용
Lake 적재	스토리지 권한·용량	싸다
검증(validate)	나쁜 문서 · 빈 버킷	싸다 — 읽기만
임베딩·적재	모델 서버(Ollama) 멈춤 API 한도 · 차원 불일치	비싸다 — 여기가 핵심

임베딩·적재가 핵심 — 여기가 비싸기 때문에 앞 단계와 반드시 떼어 둔다

3. 데이터 전달

쪽지에는 「창고 3번 선반」만 — 짐은 창고에

XCom 으로 넘기기

메타데이터 DB 에 저장 — 수 KB 가 한계

- DataFrame 을 넣으면 DB 가 붓는다
- 재시도할 때 앞 태스크도 다시 돌아야
- 적합한 것: 건수·플래그·경로 문자열

VS

스토리지 경로 넘기기

MinIO·파일시스템 — 크기 한계가 사실상 없다

- 실제 데이터는 스토리지에
- 경로만 알면 이어서 재시도
- 넘기는 것은 언제나 「어디에 있는지」

4. 컨테이너 간 네트워크

같은 Qdrant 인데 주소가 다르다

1. **같은 도커 네트워크에서는 컨테이너 이름이 곧 주소**
   Airflow 태스크 안 → http://qdrant:6333 · rag-minio:9000 · http://ollama:11434

2. **내 PC 터미널에서는 localhost**
   http://localhost:6333 · localhost:9000 — 8-4 Streamlit 도 내 PC 에서 돈다

3. **7주차 모듈은 주소를 환경변수로 받는다**
   QDRANT_URL · MINIO_ENDPOINT · OLLAMA_HOST — 없으면 localhost

4. **docker run 으로 띄운 셋은 다른 네트워크**
   옮기지 않고 docker network connect 로 Airflow 네트워크에 붙인다

주소를 가르는 질문

어디서
실행되나

Spark 는 왜 오늘 안 돌리나

1. **실무는 SparkSubmitOperator**
   대용량 변환은 Spark 클러스터에 제출한다
2. **오늘 환경은 단일 노트북**
   Airflow 일곱 컨테이너에 Ollama 모델까지 — Spark 를 더하면 메모리가 부족
3. **문서는 7주차 build_kb.py 로**
   Spark 제출은 구조만 확인한다
4. **태스크 경계와 의존성 배선은 똑같다**
   안에서 무엇을 실행하느냐만 다르다

정리

이 1시간에서 꼭 가져갈 것

1. 실패했을 때 그 단계만 다시 돌리고 싶은 지점이 태스크 경계다
2. 비싼 일과 싼 일을 한 태스크에 섞지 않는다
3. 태스크 사이에는 데이터가 아니라 경로를 넘긴다
4. 같은 도커 네트워크의 컨테이너끼리는 컨테이너 이름이 주소다
5. 실행 위치(컨테이너 안/밖)에 따라 같은 서비스의 주소가 달라진다
6. 변환을 파이썬으로 하든 Spark 로 제출하든 DAG 모양은 같다

8-3 · 이론 1H 마무리

다음은 수집→Lake→변환 태스크 구현 (1.5H)

7주차 컨테이너 셋을 Airflow 네트워크에 붙이고, collect · validate · build_index · report 를 채워 돌립니다

- ✓ **docker network connect 로 셋 붙이기**
- ✓ **.env 에 패키지 · 주소 네 줄**
- ✓ **7주차 모듈 셋을 dags/ 에 복사**
- ✓ **짧은 문서로 validate 가 막는지 확인**

준비할 것

실습점검 1~3단계 · 8-2 분해안